



Apuntes I2

Temario

- Clase 5 y 6 - SQL desde Inner Joins
 - Clase 7 - Dependencias
 - Clase 8 - Lógica en la Base de Datos
 - Clase 9 - ACID, Transacciones y Locks
 - Clase 10 - Arquitectura de Sistemas y Recuperación de fallas
-

1. SQL Avanzado:

INNER JOIN

Definición: Un **INNER JOIN** devuelve solo las filas que tienen coincidencias en ambas tablas.

Sintaxis:

```
SELECT columns  
FROM table1 INNER JOIN table2  
ON table1.common_column = table2.common_column;
```

Ejemplo:

```
SELECT employees.name, departments.department_name  
FROM employees INNER JOIN departments  
ON employees.department_id = departments.department_id;
```

Tablas de Ejemplo:

employees			departments	
employee_id	name	department_id	department_id	department_name

1	John	101	101	HR
2	Alice	102	102	IT
3	Bob	103	103	Marketing
4	Eve	NULL	104	Sales

Resultado del INNER JOIN :

name	department_name
John	HR
Alice	IT
Bob	Marketing

LEFT JOIN

Definición: Un **LEFT JOIN** devuelve todas las filas de la tabla de la izquierda (table1) y las filas coincidentes de la tabla de la derecha (table2). Si no hay coincidencia, se llenan con **NULL**.

Ejemplo:

```
SELECT employees.name, departments.department_name
FROM employees LEFT JOIN departments
ON employees.department_id = departments.department_id;
```

Resultado del LEFT JOIN :

name	department_name
John	HR
Alice	IT
Bob	Marketing
Eve	NULL

RIGHT JOIN

Definición: Un **RIGHT JOIN** devuelve todas las filas de la tabla de la derecha (table2) y las filas coincidentes de la tabla de la izquierda (table1). Si no hay coincidencia, se llenan con **NULL**.

Ejemplo:

```
SELECT employees.name, departments.department_name
FROM employees RIGHT JOIN departments
ON employees.department_id = departments.department_id;
```

Resultado del **RIGHT JOIN** :

name	department_name
John	HR
Alice	IT
Bob	Marketing
NULL	Sales

FULL JOIN

Definición: Un **FULL JOIN** devuelve todas las filas cuando hay una coincidencia en una de las tablas. Devuelve **NULL** para las filas sin coincidencia en la otra tabla.

Ejemplo:

```
SELECT employees.name, departments.department_name
FROM employees
FULL JOIN departments
ON employees.department_id = departments.department_id;
```

Resultado del **FULL JOIN** :

name	department_name
John	HR
Alice	IT
Bob	Marketing
Eve	NULL
NULL	Sales

2. Formas Normales y Dependencias

Dependencias Funcionales

Una dependencia funcional en una relación **R** es una restricción entre dos conjuntos de atributos.

Si **X** y **Y** son conjuntos de atributos de **R**, **X** determina **Y** (denotado $X \rightarrow Y$)

Si para cada par de tuplas **t1** y **t2** en **R**, si $t1[X] = t2[X]$, entonces $t1[Y] = t2[Y]$

Dependencias Funcionales - Ejemplo

Considera la tabla **Guías** :

gid	nombre	score	horas	valorHora
1	Juan	8	40	18000
2	Johanna	8	30	18000
3	Cristian	5	30	15000
4	Pedro	8	32	18000

Dependencia funcional: $score \rightarrow valorHora$

Esto indica que el valor de **valorHora** está determinado por **score**.

Formas Normales

Primera Forma Normal (1NF)

Una relación está en 1NF si y solo si todos los atributos son atómicos (no divisibles) y cada registro es único.

Ejemplo de relación NO en 1NF:

ID cliente	Cliente	Transacciones
1	Isaac	(1, 2003-10-14, 100.000), (2, 2003-10-15, 150.000)

Para transformar a 1NF, se descompone en varias filas:

ID cliente	Cliente	ID transacción	Fecha	Cantidad
1	Isaac	1	2003-10-14	100.000
1	Isaac	2	2003-10-15	150.000

Segunda Forma Normal (2NF)

Una relación está en 2NF si **está en 1NF** y todos **los atributos no clave son completamente dependientes de la clave primaria**, es decir, no hay dependencias parciales.

Ejemplo de relación NO en 2NF:

Fabricante	Modelo	País Fabricante
Forte	X-Prime	Italia
Forte	Ultraclean	Italia
Hoch	Toothmaster	Alemania

Dependencia funcional parcial: **Fabricante → País Fabricante**

Descomposición para lograr 2NF:

Tabla 1:

Fabricante	Modelo
Forte	X-Prime
Forte	Ultraclean
Hoch	Toothmaster

Tabla 2:

Fabricante	País Fabricante
Forte	Italia
Hoch	Alemania

Tercera Forma Normal (3NF)

Una relación está en 3NF si está en 2NF y ningún atributo no clave es transitivamente dependiente de la clave primaria.

Ejemplo de relación NO en 3NF:

Fabricante	Modelo	País Fabricante	Continente
Forte	X-Prime	Italia	Europa

Hoch	Toothmaster	Alemania	Europa
------	-------------	----------	--------

Dependencia transitiva: País Fabricante → Continente

Descomposición para lograr 3NF:

Tabla 1:

Fabricante	Modelo	País Fabricante
Forte	X-Prime	Italia
Hoch	Toothmaster	Alemania

Tabla 2:

País Fabricante	Continente
Italia	Europa
Alemania	Europa

Forma Normal de Boyce-Codd (BCNF)

Una relación está en BCNF si para toda dependencia funcional $X \rightarrow Y$, X es una **SUPERLLAVE**.

Esto es más restrictivo que 3NF.

Ejemplo de relación NO en BCNF:

Curso	Profesor	Aula
CS101	Dr. Smith	A101
CS101	Dr. Brown	A102

Dependencia funcional: Profesor → Aula, pero Profesor no es superllave.

Para convertir a BCNF, descomponer la tabla:

Tabla 1:

Curso	Profesor
CS101	Dr. Smith
CS102	Dr. Brown

Tabla 2:

Profesor	Aula
----------	------

Dr. Smith	A101
Dr. Brown	A102

Resumen de Dependencias y Formas Normales

Forma Normal	Condiciones
1NF	Atributos atómicos y registros únicos
2NF	En 1NF y sin dependencias parciales
3NF	En 2NF y sin dependencias transitivas
BCNF	En 3NF y cada determinante es una superllave

EXPLICACION LLAVES:

- “Una **superllave** es cualquier conjunto de uno o más atributos que identifica de manera única una tupla en una relación. En otras palabras:

“No puede haber dos filas en la tabla que tengan los mismos valores para los atributos de la superllave.”

Clave Primaria y Clave Candidata

- **Clave candidata:** Es un conjunto mínimo de atributos que puede identificar de manera única una tupla en una relación. Una relación puede tener múltiples claves candidatas.
- **Clave primaria:** Es una clave candidata elegida por el diseñador de la base de datos para identificar de manera única las tuplas de una relación. Solo puede haber una clave primaria por tabla.
- **Superllave:** Cualquier combinación de atributos que incluya una clave candidata. Por lo tanto.

“Todas las claves candidatas son superllaves, pero no todas las superllaves son claves candidatas.”

Ejemplo

Consideremos la siguiente tabla de **Estudiantes** :

student_id	first_name	last_name	email	phone_number
------------	------------	-----------	-------	--------------

1	John	Doe	john.doe@mail.com	1234567890
2	Jane	Smith	jane.smith@mail.com	2345678901
3	Alice	Brown	alice.brown@mail.com	3456789012

Claves Candidatas

- `student_id` : Identifica de manera única a cada estudiante.
- `email` : También puede identificar de manera única a cada estudiante si cada correo es único.

Superllaves

Cualquier combinación de atributos que incluya una clave candidata es una superllave. Por ejemplo:

- `{student_id}`
- `{email}`
- `{student_id, email}`
- `{student_id, first_name}`
- `{email, last_name, phone_number}`
- etc.

Clave primaria:

Clave candidata elegida para identificar de manera única las tuplas en una relación. Por ejemplo:

- `{student_id}`

3. Lógica en Base de Datos

Vistas

Definición y Funcionamiento

- Las vistas son consultas guardadas en la base de datos que pueden ser reutilizadas.

- No almacenan datos físicamente; se reescriben las consultas reemplazando la vista por la consulta original.

Ventajas de Usar Vistas

- **Abstracción:** Simplifican consultas grandes y evitan la repetición de consultas frecuentes.
- **Mantenibilidad:** Más fácil manejar vistas que datos duplicados o redundantes.

Ejemplo de Vistas

```
CREATE VIEW PelEval AS
SELECT pelicula, director, AVG(eval) AS promedio
FROM Evaluaciones
GROUP BY pelicula, director;
```

Vistas Materializadas

- A diferencia de las vistas estándar, las vistas materializadas almacenan datos físicamente.
- Requieren espacio adicional y deben mantenerse actualizadas con comandos como `REFRESH MATERIALIZED VIEW`.

Ejemplo de Vistas Materializadas

```
CREATE MATERIALIZED VIEW PelEval AS
SELECT pelicula, director, AVG(eval) AS promedio
FROM Evaluaciones
GROUP BY pelicula, director;
```

Actualizar Vistas Materializadas

```
REFRESH MATERIALIZED VIEW PelEval;
```

- Podemos automatizar el esto en la DB? → Sí, con **TRIGGERS**

Triggers

Definición y Funcionamiento

- Los triggers son procedimientos que se ejecutan automáticamente en respuesta a ciertos eventos en la base de datos.
- Pueden ser ejecutados antes o después de eventos como INSERT, UPDATE, DELETE.

Tipos de Triggers

- **BEFORE:** Antes de la operación.
- **AFTER:** Después de la operación.
- **INSTEAD OF:** En lugar de la operación.
- Ej: BEFORE CREATE

Observacion: Se ejecuta el trigger por

1. cada FILA afectada por el evento: **FOR EACH ROW**
2. 1 vez por evento: **FOR EACH STATEMENT**

Ejemplo de Trigger

Reducir el stock de un producto cada vez que se crea una venta.

```
CREATE TRIGGER reducir_stock
AFTER INSERT ON Ventas
FOR EACH ROW
BEGIN
    UPDATE Productos
    SET stock = Productos.stock - 1
    WHERE NEW.id_productos = Productos.id;
END;
```

Ejemplo de Actualizar Vistas Materializadas

```
CREATE TRIGGER refresh_eval
AFTER CREATE ON Evaluaciones
FOR EACH STATEMENT
BEGIN
    REFRESH MATERIALIZED VIEW PelEval
END
```

Ejemplo mas completo:

Supongamos que tenemos dos tablas: **Productos** y **Ventas** .

- **Productos** : Almacena información sobre los productos y su stock disponible.
- **Ventas** : Almacena información sobre las ventas realizadas.

Estructura de las Tablas

Tabla Productos

```
CREATE TABLE Productos (  
  id_producto SERIAL PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  precio DECIMAL(10, 2) NOT NULL,  
  stock INT NOT NULL  
);
```

Tabla Ventas

```
CREATE TABLE Ventas (  
  id_venta SERIAL PRIMARY KEY,  
  id_producto INT NOT NULL,  
  cantidad INT NOT NULL,  
  fecha DATE NOT NULL,  
  FOREIGN KEY (id_producto) REFERENCES Productos(id_producto)  
);
```

Crear el Trigger

Queremos que cada vez que se inserte una nueva venta en la tabla **Ventas** , se reduzca automáticamente el stock del producto correspondiente en la tabla **Productos** .

```
CREATE OR REPLACE FUNCTION reducir_stock() RETURNS TRIGGER AS $$  
BEGIN  
  UPDATE Productos  
  SET stock = stock - NEW.cantidad  
  WHERE id_producto = NEW.id_producto;  
  
  -- Validacion de stock > 0:  
  IF (SELECT stock FROM Productos WHERE id_producto=NEW.id_producto) < 0  
  THEN  
    RAISE EXCEPTION 'Stock insuficiente %', NEW.id_producto;
```

```
END IF;  
  
RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

Explicación de la Función

- **UPDATE Productos SET stock = stock - NEW.cantidad:** Actualiza el stock del producto restando la cantidad vendida.
- **WHERE id_producto = NEW.id_producto:** Especifica el producto a actualizar, utilizando `NEW.id_producto` que es el valor del producto en la nueva fila de `Ventas`.
- **RETURN NEW:** Devuelve la nueva fila insertada en la tabla `Ventas`.

Paso 2: Crear el Trigger que Llama a la Función

Ahora creamos el trigger que se ejecutará después de cada inserción en la tabla `Ventas`.

```
CREATE TRIGGER reducir_stock_trigger  
AFTER INSERT ON Ventas  
FOR EACH ROW  
EXECUTE FUNCTION reducir_stock();
```

Ejemplo de Uso

1. Insertar Productos

```
INSERT INTO Productos (nombre, precio, stock) VALUES ('Producto A', 100.00, 50);  
INSERT INTO Productos (nombre, precio, stock) VALUES ('Producto B', 200.00, 30);
```

2. Insertar una Venta y Activar el Trigger

```
INSERT INTO Ventas (id_producto, cantidad, fecha) VALUES (1, 5, '2024-05-28');
```

Después de insertar esta venta, el stock del `Producto A` se reducirá automáticamente de 50 a 45.

Stored Procedures

Definición y Funcionamiento

- Los stored procedures son conjuntos de comandos SQL que se almacenan en la base de datos y pueden ser ejecutados como una unidad.

Ventajas de Usar Stored Procedures

- **Velocidad:** Optimizadas para ejecución rápida.
- **Seguridad:** Controlan el acceso a datos y lógica.
- **Reutilizabilidad:** Una vez definidos, pueden ser reutilizados múltiples veces.

Desventaja principal:

- **Dificultad o imposibilidad de migrar a otra base de datos**
- El testing sea mucho más difícil
- hace que sea más difícil mantener el código, porque SQL tiende a ser menos legible a medida que se vuelve más complejo.

Ejemplo de Stored Procedure

Insertar una nueva persona en la tabla

`personas` .

```
CREATE OR REPLACE FUNCTION insertar_persona (rut varchar, nombre varchar,
apellido varchar)
RETURNS void AS $$

--DECLARE
-- Aquí irían las declaraciones de variables

BEGIN
  INSERT INTO personas VALUES (rut, nombre, apellido);
END;
$$ LANGUAGE plpgsql;
```

Ahora para usarla:

```
SELECT insertar_persona('11.111.111-1', 'pepito', 'los palotes')
```

4. ACID, Transacciones y Locks

Transacciones

- Una transacción es una secuencia de operaciones que modifican o consultan la base de datos.
- Su principal objetivo es garantizar la integridad de los datos y la consistencia del estado de la base de datos, incluso en presencia de fallos y errores.
- Las transacciones tienen varias ventajas, como:
 - la integridad de los datos
 - la consistencia
 - el aislamiento
 - la durabilidad.
- Sin embargo, también tienen desventajas, como:
 - la complejidad en el manejo,
 - el rendimiento
 - el uso de recursos.

Transacciones

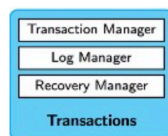
Ventajas	Desventajas
Integridad de los datos - Asegura que todos los cambios en la base de datos son correctos y se preserva la consistencia, incluso ante errores o fallos del sistema.	Complejidad en el manejo - Requiere mecanismos sofisticados de control de concurrencia y recuperación, lo que puede complicar el diseño y el mantenimiento de la base de datos.
Consistencia - Garantiza que la base de datos siempre refleje un estado válido según las reglas definidas, gracias a la ejecución completa de las transacciones.	Rendimiento - Las transacciones, especialmente cuando son grandes o complejas, pueden afectar el rendimiento debido al bloqueo de recursos y a la gestión del control de concurrencia.
Aislamiento - Permite que múltiples transacciones se ejecuten simultáneamente sin interferir entre ellas, manteniendo la exactitud de los datos.	Uso de recursos - Puede requerir un uso intensivo de recursos, como memoria y procesador, para gestionar el estado y la sincronización de las transacciones.
Durabilidad - Una vez que una transacción se completa, sus efectos son permanentes, incluso en caso de fallo del sistema.	Tiempo de espera - Los bloqueos necesarios para mantener el aislamiento pueden llevar a tiempos de espera o a deadlocks que deben ser gestionados cuidadosamente.

Propiedades ACID

Las transacciones deben cumplir con las propiedades ACID para asegurar su correcto funcionamiento:

1. **Atomicidad:** O se ejecutan todas las operaciones de la transacción, o no se ejecuta ninguna.
2. **Consistencia:** Cada transacción preserva la consistencia de la base de datos (restricciones de integridad, etc.).
3. **Isolation:** Cada transacción debe ejecutarse como si se estuviera ejecutando sola, de forma aislada.
4. **Durabilidad:** Los cambios que hace cada transacción son permanentes en el tiempo, independiente de cualquier tipo de falla.

Módulo de Transacciones



Transaction Manager se encarga de asegurar Isolation y Consistency

Log y Recovery Manager se encargan de asegurar Atomicity y Durability

Problemas de Concurrency (Acceder al mismo atributo simultáneamente):

Transacciones

Transferencia doble

Pero el acceso es CONCURRENTE opción 1

Proceso Alice	Proceso Bob	Saldo Cuenta A & B	Saldo Cuenta C
READ(saldoAB, x)		1000	1000
WRITE(saldoAB, x - 100)		900	
	READ(saldoAB, y)	900	
	WRITE(saldoAB, y - 200)	700	
	READ(saldoC, y)		1000
	WRITE(saldoC, y + 200)		1200
READ(saldoC, x)			1200
WRITE(saldoC, x + 100)			1300

Transacciones

Transferencia doble

Pero el acceso es CONCURRENTE opción 2

Proceso Alice	Proceso Bob	Saldo Cuenta A & B	Saldo Cuenta C
READ(saldoAB, x)		1000	1000
WRITE(saldoAB, x - 100)		900	
READ(saldoC, x)			x = 1000
	READ(saldoAB, y)	900	
	WRITE(saldoAB, y - 200)	700	
	READ(saldoC, y)		1000
	WRITE(saldoC, y + 200)		1200
WRITE(saldoC, x + 100)			1100

¡Se perdieron 200!

Si no se cumple ACID:

Sin Atomicidad y Durabilidad:

1. Problemas con transacciones a la mitad
2. Cambios en las transacciones no se ven reflejados en la DB

Sin Consistencia:

1. La DB Viola las restricciones momentaneamente
2. Al ejecutar transacciones, queda la DB que no cumple restricciones

Sin Isolacion:

1. El sistema de DB planifica orden de las operaciones
2. Resultados no es igual a haber corridos transiciones en serie

Transacciones en SQL:

```
START TRANSACTION
```

```
UPDATE cuentas
```

```
SET saldo = saldo - v
```

```
WHERE cid = 1
```

```
UPDATE cuentas
```

```
SET saldo = saldo + v
```

```
WHERE cid = 2
```

```
COMMIT
```

Transacciones en SQL :

- START TRANSACTION y COMMIT nos permiten agrupar operaciones en una sola transacción

Operaciones:

1. Read(x): Lectura del elemento x
2. Write(x): Escritura del elemento x

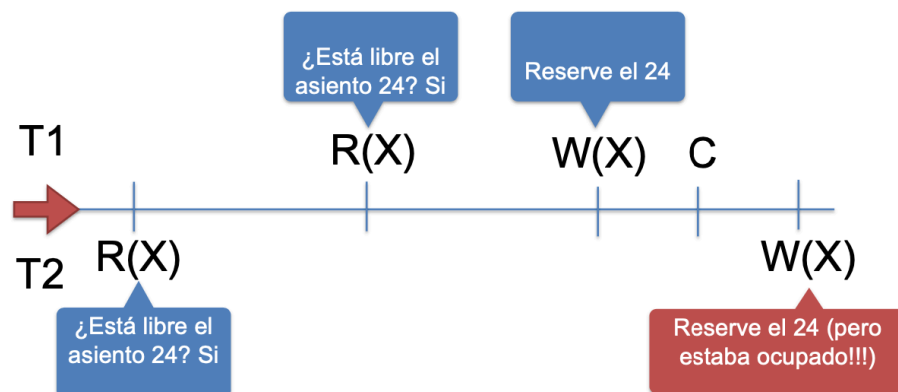
3. Abort: Abortar Transacción
4. Commit: Finalizar Transacción

Ejemplos de Conflictos

- **Lecturas Sucias (Write - Read):** Ocurren cuando una transacción accede a datos que han sido modificados por otra transacción que aún no se ha terminado.
- **Lecturas No Repetibles (Read - Write):** Ocurren cuando una fila es leída dos veces y el valor cambia entre ambas lecturas.
- **Actualización Perdida o Reescritura de Datos Temporales (Write - Write):** Ocurren cuando dos transacciones que intentan actualizar la misma fila son procesadas en un tiempo que permite que una de las actualizaciones sobrescriba a la otra.

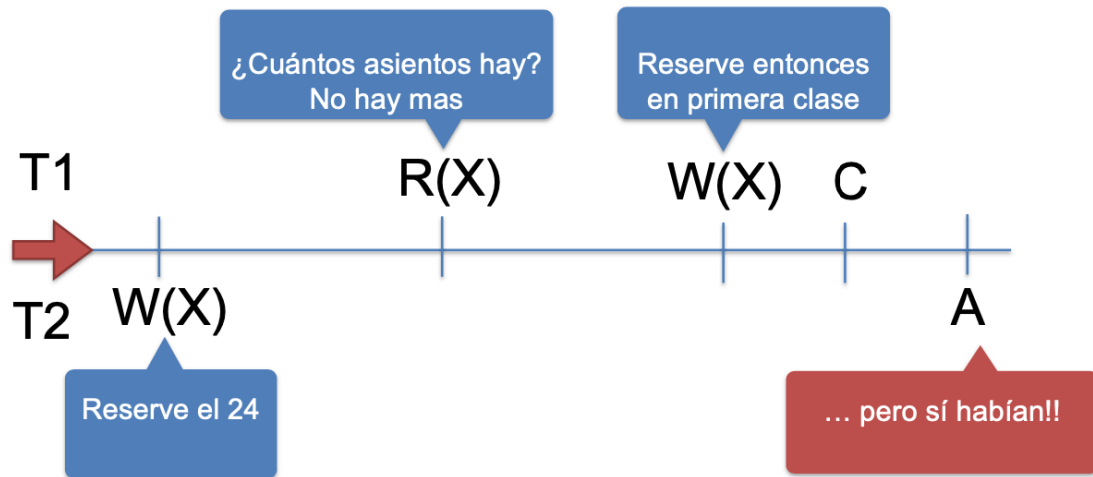
Ejemplo de actualización perdida (WW)

“Una transacción sobrescribe los datos que otra tx ya había escrito”



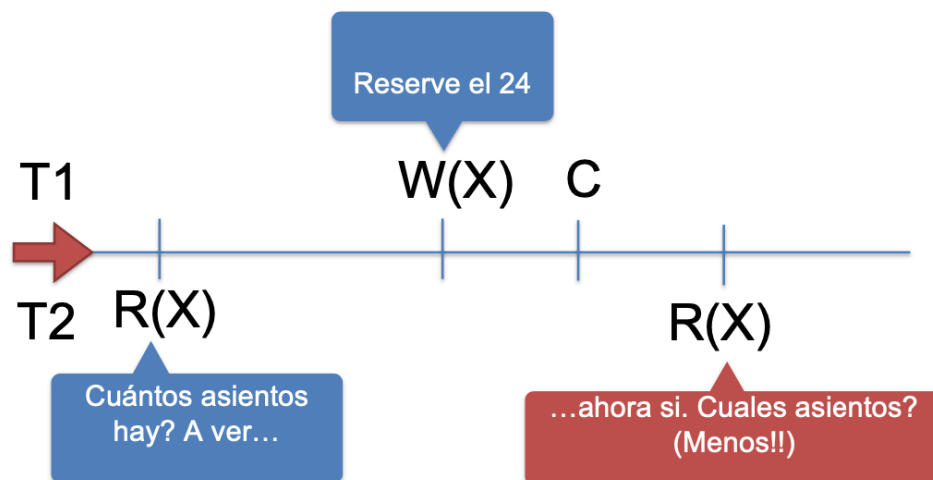
Ejemplo de lectura sucia (WR)

“Una tx. lee lo que otra tx escribió pero no se había confirmado aún.”



Ejemplo de lectura no repetible (RW)

“Una tx. sobrescribe un dato que otra ya había leído antes pero no había confirmado.”



Ejemplos:

--Lecturas Sucias:

READ(A,x)

WRITE(A,x)

--Lecturas No Repetibles:

READ(A,x)

WRITE(A,x)

--Actualización Perdida o Reescritura de Datos Temporales:

WRITE(A,x)

WRITE(A,x)

Schedules (S)

- Un schedule (programa de ejecución) es una secuencia de operaciones primitivas de una o más transacciones.
- Las operaciones pueden ser de lectura (R) o escritura (W).
- Un schedule es **Serial** si no hay intercalación entre las acciones.

Schedule Serial

Un schedule serial

T1	T2
READ(A,x)	
x:= x + 100	
WRITE(A,x)	
READ(B,x)	
x:= x + 200	
WRITE(B,x)	
	READ(A,y)
	y:= y * 2
	WRITE(A,y)
	READ(B,y)
	y:= y * 3
	WRITE(B,y)

- Un schedule es **Serializable** si existe algún schedule serial con las mismas transacciones, tal que el resultado de S y S' es el mismo para todo estado inicial

de la BD.

Schedule Serializable

T1	T2
READ(A,x)	
x:= x + 100	
WRITE(A,x)	
	READ(A,y)
	y:= y * 2
	WRITE(A,y)
READ(B,x)	
x:= x + 200	
WRITE(B,x)	
	READ(B,y)
	y:= y * 3
	WRITE(B,y)

Como las operaciones para A en x e y no afectan al orden de x e y de B, entonces SI es serializable

Schedule No Serializable

T1	T2
READ(A,x)	
x:= x + 100	
WRITE(A,x)	
	READ(A,y)
	y:= y * 2
	WRITE(A,y)
	READ(B,y)
	y:= y * 3
	WRITE(B,y)
READ(B,x)	
x:= x + 200	
WRITE(B,x)	

En T2, la transaccion de WRITE(B, y) realiza cambios que afectan a READ(B,x), por lo que el orden si cambian los resultados y por lo mismo el schedule No es serializable

Ejemplos de Schedules

- **Schedule Serial:**

```
READ(A,x)
x = x + 100
WRITE(A,x)
```

```
READ(B,x)
x = x + 200
WRITE(B,x)
```

- **Schedule No Serializable:**

```
READ(A,x)
x = x + 100
WRITE(A,x)
```

```
READ(B,y)
y = y * 3
WRITE(B,y)
```

```
READ(B,x)
x = x + 200
WRITE(B,x)
```

Transacciones no Conflictivas:

- Transaccion i ejecuta Read(X,t) $\rightarrow R_i(x)$
- Transaccion i ejecuta Write(X,t) $\rightarrow W_i(x)$

Acciones No Conflictivas

Las siguientes acciones son NO conflictivas para dos transacciones distintas i, j :

- $R_i(X), R_j(Y)$
- $R_i(X), W_j(Y)$ con $X \neq Y$
- $W_i(X), R_j(Y)$ con $X \neq Y$
- $W_i(X), W_j(Y)$ con $X \neq Y$

Podemos cambiarlas de orden en un **schedule**!

Permutaciones permitidas

T1	T2
READ(A)	
	READ(A)
	READ(C)
READ(B)	
WRITE(A)	
	WRITE(C)
READ(D)	

Permutaciones permitidas

T1	T2
	READ(A)
READ(A)	
READ(C)	
WRITE(A)	
	READ(B)
READ(D)	WRITE(C)

Acciones Conflictivas:

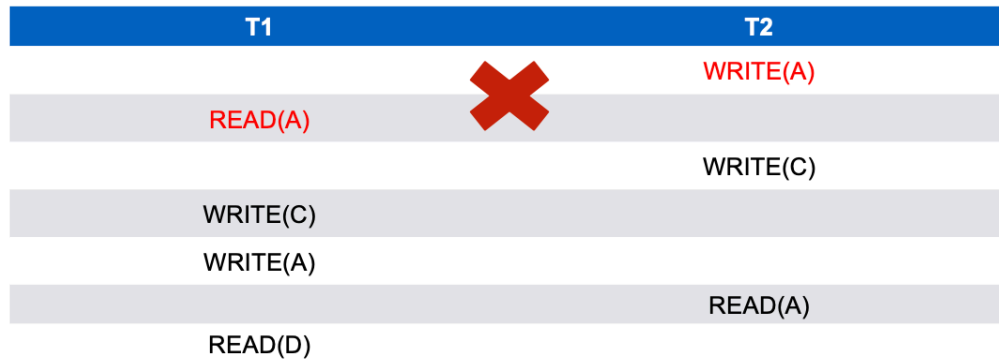
Acciones Conflictivas

Las siguientes acciones son conflictivas para dos transacciones distintas i, j :

- Lectura-Escritura ($R_i(X), W_j(X)$): Transacción i lee X , transacción j escribe X .
- Escritura-Lectura ($W_i(X), R_j(X)$): Transacción i escribe X , transacción j lee X .
- Escritura-Escritura ($W_i(X), W_j(X)$): Ambas transacciones, i y j , escriben en X .

¡No podemos cambiar su orden en un **schedule** a la ligera!

Permutaciones no permitidas



Se Pueden Permutar Operaciones Consecutivas:

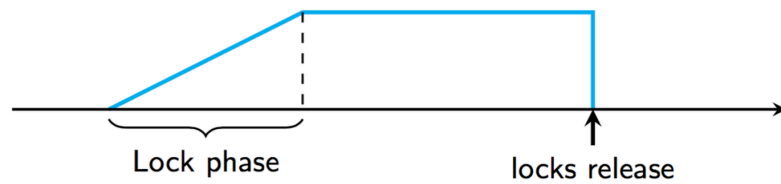
1. Par de operaciones que NO USEN EL MISMO RECURSO
2. Par de operaciones que USAN EL MISMO RECURSO PERO AMBAS DE LECTURA

Schedule es "CONFLICT SERIALIZABLE" → Si se puede transformar en uno serial usando permutaciones.

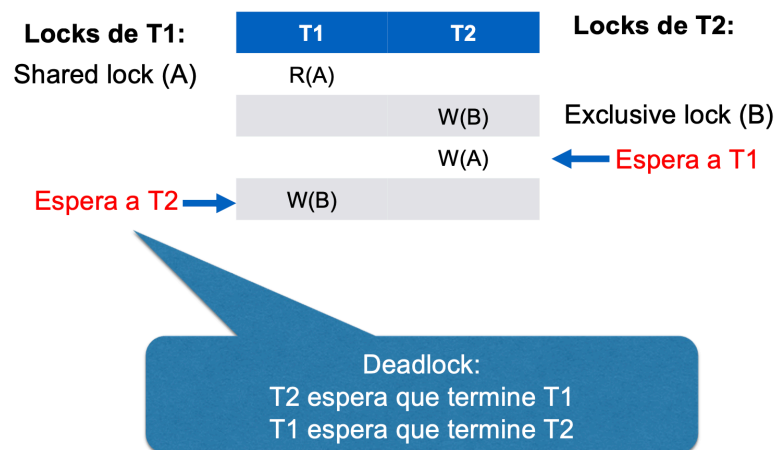
Strict 2PL (Locked Protocols)

- Regla 1: Si una Transacción T quiere leer/modificar un objeto
 - Primero pide un **shared lock / exclusive lock** sobre el objeto
 - La Transacción se suspende hasta que consigue el lock
 - Si una transacción mantiene un exclusive lock de un objeto, ninguna otra transacción puede mantener un shared o exclusive lock sobre el objeto
 - Para obtener el **exclusive lock**, no debe haber ningún lock sobre el objeto
- Regla 2:
 - Cuando la transacción se completa, libera todos los locks que mantenía

Strict 2PL.



- **Deadlocks:** en 2PL



Transacciones en SQL

1. Commit

```
START TRANSACTION;  
  SELECT a_nombre  
  FROM Actores;  
COMMIT;
```

-- Se hace autom. cuando se ejecuta una consulta --
-- O cuando se conecta a la DB --

2. Rollback

```
START TRANSACTION;
  UPDATE Actores
    SET bio = 'El mejor actor'
    WHERE nombre = 'Adrian Soto';
ROLLBACK;

-- Rollback → Deshace una transaccion
```

3. SavePoints

```
START TRANSACTION;
  UPDATE Actores
    SET bio = 'El mejor actor'
    WHERE nombre = 'Adrian Soto';
  SAVEPOINT MejorActor;
  UPDATE Actores
    SET bio = 'El peor actor'
    WHERE nombre = 'Juan Reutter';
ROLLBACK TO SAVEPOINT MejorActor;

-- Al ejecutar "ROLLBACK TO SAVEPOINT" se borra el SAVEPOINT

-- Muy util para hacer varias transacciones y verificar condiciones
```

4. Locks en Sql

T1

SELECT S.rating, MIN(S.age)
FROM Sailors AS S
WHERE S.rating = 8;

shared lock

sid	age	rating
1	34	9
2	21	8
3	16	8
4	26	10

T2

INSERT INTO Sailors AS S
VALUES (5,22,8);

shared lock

sid	age	rating
1	34	9
2	21	8
3	16	8
4	26	10
5	22	8

"fantasma"

T2 :

espera a T1

Nivel de Aislamiento (Granulidad):

Nivel de aislamiento (granularidad)

SET TRANSACTION ISOLATION LEVEL Level READ
ONLY

SET TRANSACTION ISOLATION LEVEL Level READ
WRITE

Level

- SERIALIZABLE
- REPEATABLE READ
- READ COMMITTED
- READ UNCOMMITTED

Dirty Read	Unrepeatable Read	Phantom
No	No	No
No	No	Maybe
No	Maybe	Maybe
Maybe	Maybe	Maybe

Por defecto:

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE
READ WRITE

5. Arquitectura de Sistemas y Recuperación de fallas

Almacenamiento y Jerarquía de Memoria

Disco Duro

- **Sector:** Es la unidad física mínima de almacenamiento en un disco duro, generalmente de 512 bytes o 4096 bytes en discos más modernos.
- **Página:** Unidad lógica mínima de almacenamiento que el sistema operativo gestiona, generalmente de 4 KB.
- **Clúster:** Conjunto de sectores agrupados para mejorar la eficiencia de la gestión de archivos.
- **Estructura del Disco:**
 - **Pistas:** Anillos concéntricos en el disco.
 - **Sectores:** Divisiones de las pistas.
 - **Cilindros:** Conjunto de pistas alineadas verticalmente a través de los platos del disco.

Rendimiento del Disco Duro

- **Tiempo de búsqueda:** El tiempo que tarda la cabeza de lectura/escritura en posicionarse sobre la pista correcta.
- **Retraso rotacional:** El tiempo de espera hasta que el sector deseado se posiciona bajo la cabeza de lectura/escritura.
- **Tiempo de transferencia:** El tiempo necesario para transferir los datos desde el disco a la memoria principal.

Relación entre Disco y RAM

- **Operación del DBMS:**
 - El sistema de gestión de bases de datos (DBMS) trabaja a nivel de página.

- Utiliza la RAM para almacenar temporalmente las páginas cargadas desde el disco.
- Implementa un buffer pool para gestionar estas páginas y optimizar el acceso a los datos.

Tipos de Fallas en Transacciones y Soluciones

Fallas en la Ejecución de Transacciones

- **Datos Erróneos:**

- **Problema:** Entrada de datos incorrectos debido a errores humanos o problemas de integración.
- **Soluciones:** Implementar validaciones estrictas, controles de integridad
- procesos de limpieza de datos periódicos.

- **Fallas en el Almacenamiento:**

- **Problema:** Fallos físicos en los dispositivos de almacenamiento.
- **Soluciones:** Uso de configuraciones RAID (Redundant Array of Independent Disks) para redundancia
- Copias de seguridad regulares.

- **Catástrofes:**

- **Problema:** Eventos graves como incendios, inundaciones, ataques cibernéticos, etc.
- **Soluciones:** Implementar estrategias de recuperación ante desastres
- copias de seguridad distribuidas geográficamente
- planes de continuidad del negocio.

- **Fallas del Sistema:**

- **Problema:** Interrupciones en el funcionamiento del sistema de base de datos, como fallos de software o hardware.
- **Solución:** Utilización de técnicas de recuperación mediante logs, checkpointing y reinicio del sistema para restaurar el estado consistente de la base de datos.

- Seguir escribiendo este resumen ...

6. EDD y Almacenamiento e Índices

Cómo se guarda una tabla

Supongamos una tabla $T(a \text{ int}, b \text{ text})$ con 9 tuplas

Disco Duro

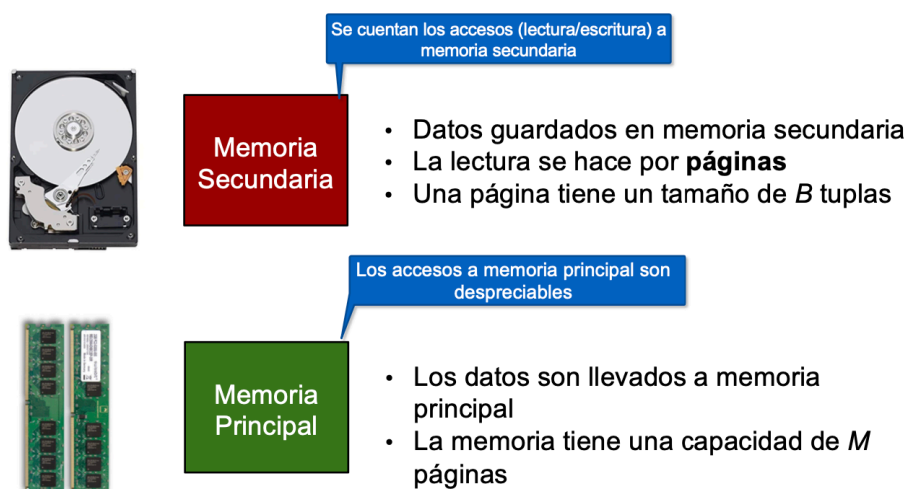
Tupla 1	Tupla 2	Tupla 3	Tupla 4	Tupla 5	Tupla 6	Tupla 7	Tupla 8	Tupla 9
Página 1			Página 2			Página 3		

- El costo mas grande en una base de datos es:
 - Traer las paginas del disco duro a la RAM
 - Objetivo: Responder las consultas haciendo la menor cantidad de lecturas al disco
- **Costo I/O:** Numero de paginas llevadas a la memoria para responder consulta

Memorias

Modelo de costos

Memoria Principal vs. Memoria Secundaria



Costo de Lecturas:

Modelo de costos



¿Cuánto cuesta leer n tuplas de memoria secundaria?

$$\left\lceil \frac{n}{B} \right\rceil$$

¿Cuántas páginas usa una relación R ?

$$\left\lceil \frac{|R|}{B} \right\rceil$$

Ejemplo de Costo de una consulta:

Costo de una consulta

Disco Duro

Tupla 1	Tupla 2	Tupla 3	Tupla 4	Tupla 5	Tupla 6	Tupla 7	Tupla 8	Tupla 9
Página 1			Página 2			Página 3		

¿Cuál es el costo en I/O de hacer la siguiente consulta?

SELECT * FROM T

$$\left\lceil \frac{|R|}{B} \right\rceil = \left\lceil \frac{9}{3} \right\rceil$$

El costo es 3, porque debo leer las 3 páginas

¿Cuál es el costo en I/O de hacer la siguiente consulta?

SELECT T.a FROM T

$$\left\lceil \frac{|R|}{B} \right\rceil = \left\lceil \frac{9}{3} \right\rceil$$

El costo nuevamente es 3, porque debo leer las 3 páginas

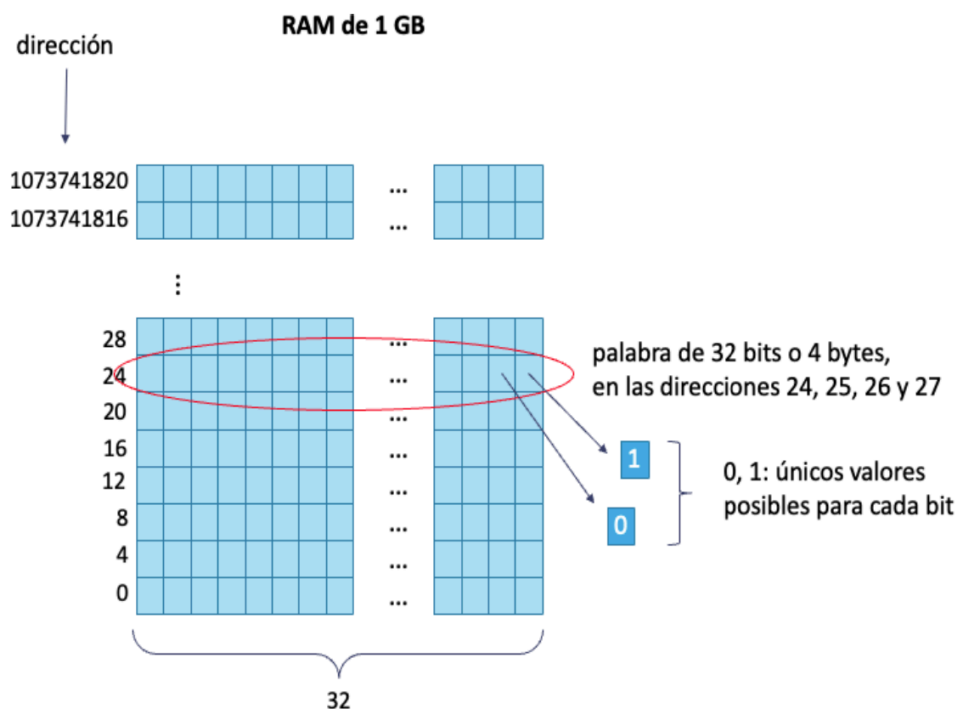
¿Cuál es el costo en I/O de hacer las siguientes consultas?

SELECT * FROM T WHERE T.a = 4
SELECT * FROM T WHERE T.a >= 4

$$\left\lceil \frac{|R|}{B} \right\rceil$$

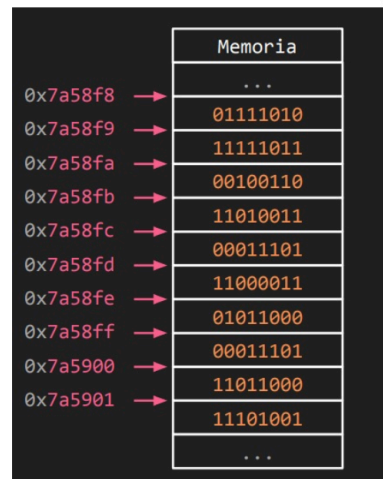
Memoria Ram

- La memoria ram es una gran matriz donde se guardan 0 y 1 (Bits)
- Se pueden acceder a esos datos mediante direcciones
- Cada fila tiene una direccion unica



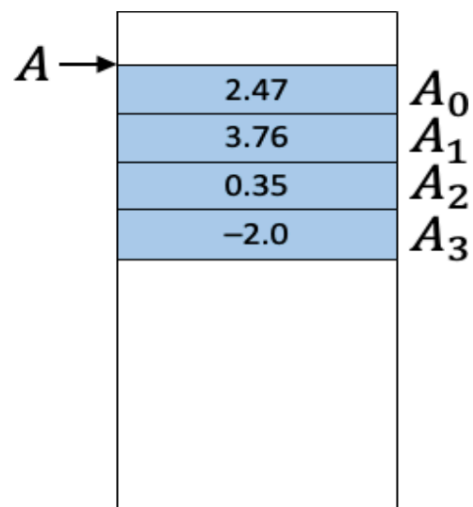
Puntero

- Es la variable cuyo **valor** es la **DIRECCION** de memoria de otra variable



Arreglo

- Un arreglo es una secuencia de celdas de largo fijo
- Almacenan valores del mismo tipo
- Es una particion de la memoria RAM



Listas ligadas

- Secuencia de celdas que tienen largo variable
- Cada celda es del mismo tamaño
- Se almacena en memoria de manera **aleatoria**, utilizando **PUNTEROS**

Diccionarios:

- Estructura de datos que cumple con:
 - Asociar un valor a una llave
 - Actualiza el valor asociado a una llave
 - Obtiene el valor asociado a una llave
 - Elimina de la estructura la asociacion llave-valor
- Es eficiente al buscar con llaves
- El orden de las llaves **NO** es relevante

Ejemplo:

Ejemplo:

En la universidad hay aproximadamente 25.000 estudiantes este año. Asignamos un $k \in \{0, \dots, 24.999\}$ a cada estudiante y usamos cada natural como índice del arreglo A.

- Rango de rut: hasta el 25.000.000
- Cantidad estudiantes: 25.000 (Mucho menos)
- Problema: Solo 1/1000 celdas del arreglo A indexado por Rut estaran ocupadas
- Nos acercamos a un conjunto de llaves K razonable con Tablas Hash

Tablas Hash

- Dado un espacio de llaves K y un natural $m > 0$, una función de hash se define como
 - $h : K \rightarrow \{0, \dots, m - 1\}$

- Dado $k \in K$, llamaremos valor de hash de k a la evaluación $h(k)$. Notemos que:
 - Una función de hash nos permite mapear un espacio de llaves a otro mas pequeño (con m razonable)
 - Una función de hash no necesariamente es inyectiva,
 -